

FirstAGI

Архитектура системы обучения с прямым вычислением аттрактора

Система учится читать, как ребёнок осваивает язык — шаг за шагом, замечая закономерности, укрепляя то, что работает, и забывая шум. Результат: модель, которая понимает контекст и умеет рассказывать.

СОДЕРЖАНИЕ

2	3	4	5
Общая архитектура	Алгоритм обучения	Система памяти	Активная фаза

КЛЮЧЕВЫЕ ХАРАКТЕРИСТИКИ

1 249 тестов в кодовой базе	**1 249 000+** токенов за тренировку	**MR = 0.926** точность после 5 проходов
435 Gen за обучение на 64 текстах	**ProvisionalBase** контроль шума	**4 уровня** творчества нарратора

ПРИНЦИПЫ РАБОТЫ

Инкрементальность
Система обучается на каждом токене отдельно, без пакетного обучения.

Экономия памяти
Слабые обобщения удаляются автоматически — память не засоряется.

Прозрачность
Каждое знание имеет метку и историю: можно объяснить любое решение.

Активность
Модель не только воспринимает — она рассказывает, задаёт вопросы, мечтает.

ХРОНОЛОГИЯ РАЗРАБОТКИ

2014	2020	2022	2024	2025	2026
Концепция BTR R&D	Ядро: Feature + Gen	Контекст + Алгоритм	Память, Сон, Бюджет	Нарратор, SpeechDriver	Активная фаза

Общая архитектура системы

От ввода текста до генерации историй — все компоненты и связи между ними

Ввод данных (Drivers)

Преобразует текст, символы или числа в поток «признаков переходов». Инвариантен к позиции в тексте.

TextDriver	CharDriver	SmartDriver	MemoryDriver	SequenceDriver
слова	символы	энтропия	воспоминания	числа

Признак (Feature)

Атомарная единица опыта — направленный переход между двумя состояниями. 5 компонент: Δяркость, Δконтраст, Δцвет, Δфрактал, направление.

Δ яркость	Δ контраст	Δ цвет	Δ фрактал	направление
лексич. ранг	сложность	оттенок	уникальность	позиция

Обобщение / Gen

Цепочка признаков, повторяющаяся в опыте. Значимость = успехи / (успехи + отказы). Гетерархия: Gen о Gen.

gen_id	feature_chain	label	significance	chain_len
уникальный ID	цепочка	слово-метка	0.0 – 1.0	глубина

Контекст + Главный алгоритм

Активные Gen = текущие «ожидания». На каждом токене: прогноз → сравнение → исход.

CONFIRMED	REFUTED	EXPLORING	COMPLETED	Context
укрепить Gen	новый Gen	исследовать	зафиксировать	буфер активных

База знаний (GeneralizationBase)

Оперативное хранилище Gen. Управляет объёмом, уровнями и вытеснением.

Provisional Base	Timescale Scheduler	Capacity Budget	GenBase DB	eviction
испытат. срок	долг/средн/операт	лимит памяти	SQLite диск	вытеснение

Консолидация (Sleep / Dream)

Периодическая «ночная работа»: слияние похожих знаний из разных доменов.

SleepScheduler	ReplayEngine	CrossDomain Detector	DreamBuffer	meta-Gen
когда спать	галлюцинация	кросс-домен	транзакции	слияние

Активная фаза (NarratorDriver)

Модель рассказывает то, что знает. WordGraph — граф слов из Gen.label.

RECALL	RETELL	COMPOSE	DREAM	GapQ
точная цитата	пересказ	сочинение	мечта	вопросы

Интерфейс и наблюдение

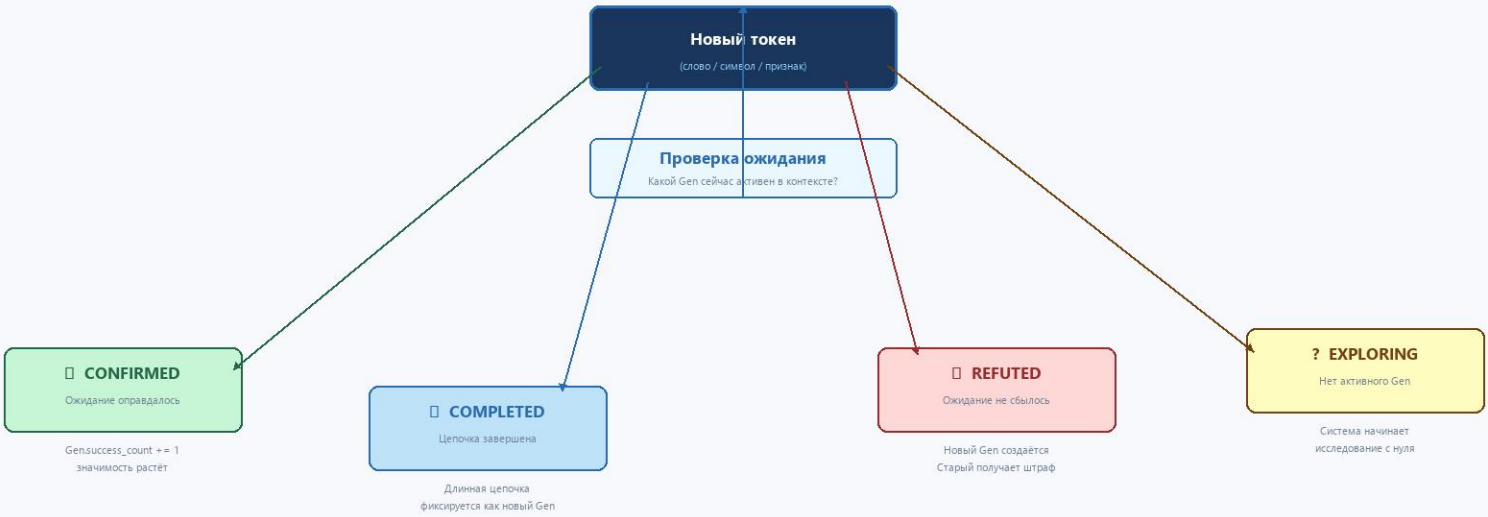
Непрерывный обучающий цикл, мониторинг, внешние источники, Telegram-бот.

Background Runner	SpeechDriver	ContextBroker	Proposal Loop	Telegram
обуч. цикл	канал B	внешн. запросы	LLM интеграция	интерфейс

Главный алгоритм обучения

Как система обрабатывает каждое новое слово (токен) — шаг за шагом

Аналогия:
Читая книгу, вы постоянно предсказываете следующее слово. Угадали — знание укрепляется. Ошиблись — мозг запоминает исключение.



ProvisionalBase

Испытательный срок для новых Gen:

- При REFUTED новый Gen помещается «на пробу»
- Параметры: min_confirmations=2, max_age=200
- Если не подтвердился — удаляется автоматически
- Решает проблему «14 000 шумовых Gen»
- max_age > длины цикла корпуса (важно!)
- Gen остаётся в base, просто под наблюдением
- Пример: Winnie corpus → base ≤ 3 000 Gen

GreedyGeneralizer

Построение цепочек признаков:

- CONFIRMED → расширяем текущую цепочку
- Цепочка НЕ сбрасывается при COMPLETED
- Только REFUTED или reset_chain() прерывают
- Чем длиннее цепочка — тем «глубже» знание
- Консолидация: общие префиксы → мета-Gen
- LONG_TERM chain 1–2, MEDIUM chain 3–4, OPER chain 5+
- Гетерархия: Gen о Gen (мета-уровни обобщений)

Формула значимости обобщения

$$significance = успехи \;/\; (успехи + отказы)$$

0.0 = никогда не работало · 1.0 = работает всегда · учитываются только активные Gen

12-шаговый пайплайн process_token()

1	**clear_completed()** Проактивная очистка завершённых Gen в начале шага	2	**tick(ProvisionalBase)** Evict Gen, не подтверждённые за max_age шагов
3	**load_context(feature)** Загрузка активных Gen (только chain=1, Priority 1+2)	4	**step(token)** Главный шаг: прогноз → сравнение → исход
5	**update_significance** Обновление success_count / stop_count	6a	**GreedyGeneralizer** Расширить / сбросить цепочку
6б	**record_vitality** Записать VitalityRecord при CONFIRMED/COMPLETED	7	**ProvisionalBase.confirm()** Подтвердить испытательный Gen
8	**CapacityBudget.enforce()** Вытеснить лишние Gen при превышении лимита	9	**AttractorEncoder** Запись траектории внимания
10	**HedonicAdapter** Гедонический сигнал (удовольствие / боль)	11	**InitiativeEngine** Обновление инициативы (когда спать / исследовать)
12	**return StepResult** Исход + matched_feature_id + контекст		

Пример: обработка предложения «Снусмумрик играет на гармошке»



EXPLORING (1-й токен)
Система видит «Снусмумрик» первый раз — нет активного Gen. Запускает исследование.

CONFIRMED (2–4-й токены)
«играет», «на», «гармошке» подтверждают ожидание. Gen.success_count растёт, значимость повышается.

REFUTED (5-й токен)
«под» не совпало с прогнозом. Создаётся новый Gen от «гармошке»→«под». Штраф старому Gen.

EXPLORING → CONFIRMED (6–7-й токены)
Снова исследование на «звёздным», затем «небом» подтверждает новый Gen. Цепочка укрепляется.

Система памяти и консолидации

Как знания хранятся, фильтруются, укрепляются и реорганизуются со временем

Аналогия:
Мозг не хранит всё подряд. Краткосрочная память переходит в долгосрочную, а ночью мозг «склеивает» похожие воспоминания из разных дней.

Уровни памяти (TimescaleScheduler)

ДОЛГОСРОЧНАЯ

chain 1–2

94% Gen · avg_sig=0.09

- базовые переходы
- буквы → слоги
- слоги → слова
- фундаментальные паттерны

СРЕДНЯЯ

chain 3–4

устойчивые фразы

- словосочетания
- типичные связки
- фрагменты предложений
- контекст сцен

ОПЕРАТИВНАЯ

chain 5+

эпизодические детали

- конкретные события
- имена+действия
- длинные цепочки
- рабочая память

CapacityBudget — ограничение объёма памяти

Рабочий диапазон: 3 000 – 47 000 Gen (5^5 – 6^6) · теоретический предел 8^8 = 16 млн Gen

Механизм вытеснения: При превышении лимита → вытесняются Gen с низкой значимостью из оперативного пула

Без CapacityBudget: 14 000+ шумовых Gen за один прогон (наблюдали на корпусе «Дом Джека»)

С CapacityBudget: база стабильна ≤ 3 000 Gen на протяжении 1 млн обучающих шагов

Интеграция: LearningSystem(capacity_budget=CapacityBudget(max_gens=8000))

Приоритет вытеснения: TimescaleScheduler.eviction_candidates() → OPERATIONAL → MEDIUM → LONG_TERM

Цикл сна и консолидации (Sleep / Dream)

Засыпание

SleepScheduler:
плато или усталость
H15-порог urgency

→

Воспроизведение

ReplayEngine:
«галлюцинация»
траекторий активации

→

Связывание

CrossDomainDetector:
общие префиксы
между доменами

→

Консолидация

Похожие Gen → мета-Gen,
память
реорганизуется

→

Пробуждение

WakeUpRestorer:
восстановление
контекста сессии

GenBaseDB — постоянное хранилище (SQLite)

gens	training_runs	run_snapshots	experiments	speech_log
все обобщения	история прогонов	снимки состояния	эксперименты	речевой лог
SQLite FTS5	SQLite FTS5	SQLite FTS5	SQLite FTS5	SQLite FTS5

Жизненный цикл обобщения (Gen)

СОЗДАН

REFUTED:
новый Gen
создаётся
track(gen_id)

→

ИСПЫТАНИЕ

ProvisionalBase:
ждёт N confirmations
max_age=200
tick() каждый шаг

→

АКТИВНЫЙ

CONFIRMED:
success_count
растёт
VitalityRecord

→

ЗРЕЛЫЙ

significance → 1.0
цепь растёт
meta-Gen
консолидация

→

УДАЛЁН

CapacityBudget
или max_age
вытесняют
base.remove()

Активная фаза: модель рассказывает

NarratorDriver — от точных цитат до свободных аналогий и творческих историй

Аналогия:
Дети любят рассказывать о том, что узнали — от точного повторения сказки до «а вот я придумал продолжение». Нарратор делает то же самое из графа знаний.

WordGraph — граф слов из знаний модели

Построение: WordGraph.build(base) — парсит Gen.label, создаёт узлы и рёбра

Узлы: Слова из Gen.label · Пример после Муми-тролля: 51 узел, 256 рёбер

Рёбра: Переходы w_i → w_{i+1} из цепочки · Вес = сумма значимостей Gen

Формат метки: TextDriver: «w0·w1·w2» · SmartTextDriver: «слово|p=N|h=X.X|role=...»

Запрос: neighbors(word, top_n=5) → топ соседей по весу рёбер

Важно: ⚠ TextDriver.label = слово-назначение (w_2), а не субъект предложения (w_1). Имена персонажей в начале предложений в граф НЕ попадают.

Четыре уровня творчества CreativityLevel

0 RECALL

Точная цитата

Жадный путь по максимальному весу рёбер. Температура = 0. Детерминированный. Остановка при повторах.

Аналогия:
слово за словом точно по памяти.

1 RETELL

Пересказ

Softmax с температурой T=0.5. Топ-3 кандидата. Остановка при посещённых соседях. Немного случайности.

Аналогия:
в целом то же самое, но своими словами.

2 COMPOSE

Свободное сочинение

Температура T=1.2. Штраф ×0.1 за повторные слова. BFS-мостики между доменами. Новые ситуации.

Аналогия:
новые детали на основе знакомых образов.

3 DREAM

Творческая аналогия

Температура T=2.5. Подмешиваются малозначимые слова. Неожиданные связи, свободные ассоциации, «сон».

Аналогия:
непредсказуемые образы из ранее несвязанных идей.

CharacterExtractor

- Ищет слова из списка известных персонажей в WordGraph
- Извлекает типичные действия (соседние глаголы)
- Строит профили: significance, actions, contexts
- is_protagonist = True при occurrence ≥ 10
- relationship(A, B) = вес рёбра A→B в графе
- Критерии кандидата: avg_sig ≥ 0.3, count ≥ 5, len ≥ 4

GapQuestionGenerator

- Находит слова с высокой значимостью, без исходящих рёбер
- Генерирует вопросы: «Что происходит с X?» «Куда идёт Y?»
- Фильтрует служебные слова (и, в, на, у, к...)
- Помогает осознать «что модель ещё не знает»
- Основа для curriculum-обучения с подсказками
- Фильтр: len(word) ≥ 4, нет исходящих рёбер в WordGraph

NarratorBridge — онлайн-обновление графа в процессе обучения

CONFIRMED / COMPLETED → on_step() усиливает веса рёбер WordGraph в реальном времени

REFUTED / EXPLORING → регистрирует пробел знания → recent_gaps() → будущие вопросы

Паттерн: bridge.on_step(tok.label, result.outcome.name) после каждого process_token()

Полный пайплайн активной фазы + пример

GeneralizationBase (база обобщений) → **WordGraph.build()** (граф слов) → **NarratorDriver** (генератор) → **NarrativeResult** (текст + вопросы) → **NarratorBridge** (обратная связь)

Пример генерации из обученной модели (корпус «Муми-тролль», 5 прогонов):

«Снусмумрика занят и испугался длинная всем сумочку в муми доле тихая стоит очень»

Детерминированный жадный путь: каждый раз одинаковый результат

«Снусмумрика занят и испугался доме очень длинная всем сумочку в муми доле»

Небольшое отклонение от «идеального» пути, суть та же

«Занят и испугался длинная всем сумочку всю снусмумрика тепло у реки вкусной»

Новые комбинации знакомых слов, BFS-мостики между темами

«Снусмумрика всю долгую и испугался тепло вкусной её тихая стоит приходит длинная»

Неожиданные ассоциации, малозначимые слова, творческий «сон»